

VISVESVARAYA TECHNOLOGICAL UNIVERSITY
BELAGAVI-590018



“A MINI PROJECT REPORT”
(Subject Code: 18CSMP68)
ON
“DELICATESSEN”

Submitted in partial fulfillment for the requirements for the Award of Degree of

BACHELOR OF ENGINEERING IN
INFORMATION SCIENCE AND ENGINEERING BY

JANVEE DIXIT (1EP18CS037)

MOTATI DEVNATH REDDY (1EP18CS059)

UNDER THE GUIDANCE OF

Asst.Prof. Manimegalai A
DEPT. Of CSE, EPCET



Department of Computer Science and Engineering
EAST POINT COLLEGE OF ENGINEERING AND TECHNOLOGY

Jnana Prabha Campus, Bidarahalli,

Bengaluru – 560 049.

2020-2021

EAST POINT COLLEGE OF ENGINEERING AND TECHNOLOGY

(Affiliated to Visvesvaraya Technological University, Belagavi)

Bangalore-560049



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CERTIFICATE

This is to certify that the **Mini Project Work** entitled “**DELICATESSEN**” is a bonafide work carried out by **JANVEE DIXIT**, bearing USN **1EP18CS037**, **MOTATI DEVNATH REDDY**, bearing USN **1EP18CS059**, in partial fulfillment for the award of **Bachelor of Engineering in Information Science and Engineering** under **Visvesvaraya Technological University, Belgaum** during the year **2020- 2021**. It is certified that all the corrections/suggestions indicated in the Internal Assessment have been incorporated in the report and submitted in the department library. This Mini Project report has been approved as it satisfies the academic requirements in respect of Mini Project Work prescribed for the award of the said degree.

GUIDE

Prof. Manimegalai A

Asst. Professor

HOD

Dr. T.K. Sateesh

Prof & Head

PRINCIPAL

Dr. T.K. Sateesh

Principal

External Examiners

Date and sign

1)

2)

ACKNOWLEDGEMENT

Any achievement, be it scholastic or otherwise does not depend solely on the individual efforts but on the guidance, encouragement and cooperation of intellectuals, elders and friends. We would like to take this opportunity to thank them all.

First and foremost, we would like to express our sincere regards and thanks to **Mr. Pramod Gowda** and **Mr. Rajiv Gowda**, CEOs East Point Group of Institutions, Bangalore, for providing necessary infrastructure and creating good environment.

We express our gratitude to **Dr. T.K. Sateesh**, Principal, EPCET who has always been a great source of inspiration.

We express our sincere regards and thanks to **Dr. T.K. Sateesh**, Professor and Head, Department of Computer Science and Engineering, EPCET, Bangalore, for his encouragement and support.

We are grateful to acknowledge the guidance and encouragement given to us by **Prof. Manimegalai** Assistant Professor, Department of Computer Science and Engineering, EPCET, Bangalore, who has rendered a valuable assistance.

We also extend our thanks to all the faculties of the **Department of Computer Science and Engineering, EPCET**, Bangalore, who have encouraged us throughout the course of the Mini Project Work.

Last, but not the least, we would like to thank our family and friends for their inputs to improve the Mimi Project works.

Name: **JANVEE DIXIT** USN: **1EP18CS037**

Name: **MOTATI DEVNATH REDDY** USN: **1EP18CS059**

ABSTRACT

Food is not simply stuff that people eat when their hungry. Food is awesome, delicious, inventive, colorful, inspiring, and so much more. A mutual love of food can bring different people together and can make the saddest person feel a little better without even meaning to.

Food is better when we are able to savour the taste and feel each bite slowly but due to the fast moving world and lack of time in a person's life this experience is often neglected or rather missed by most of us.

The idea behind "**DELICATESSEN**" app was to reduce or eliminate the wait time for food by pre ordering and saving the waiting time to allow more time for the actual purpose which is to eat food. The word Delicatessen comes from dutch which means a shop with ready made food or a restaurant.

The app provides a user friendly interface which brings both the chef and customer more close and provide a seamless experience of ordering food.

The app also provides real-time updates about the dishes available at the college canteen. The platform used to implement this application is Android Studio.

CONTENTS

Chapter No.	Topics	Page No.
1	Introduction 1.1 Introduction to the Project. 1.2 Organization of the Project. 1.3 Proposed System.	1
2	Literature Survey	4
3	Requirement Specifications 3.1 Introduction. 3.2 System Architecture. 3.3 Functional Requirements. 3.4 Non-Functional Requirements. 3.4.1 User Interface. 3.4.2 Software Requirements. 3.4.3 Hardware Requirements. 3.5 System Model.	7
4	System Design 4.1 Data Models. 4.2 Algorithmic Design. 4.3 User Interface Design.	19
5	Software Test Specifications 5.1 Goals and Major Constraints. 5.2 Testing Strategies. 5.2.1 Unit Testing. 5.2.2 Integration Testing. 5.3.3 System Testing	38
6	Conclusion	43
	References	44

LIST OF TABLES

Table No.	Table Name	Page No.
3.1	Functional Requirement	10
5.1	Unit Test	39
5.2	Test Case	41

LIST OF FIGURES

Figure No.	Figure Name	Page No.
3.1	System Architecture	8
3.2	User Interface	11
3.3	XML	12
3.4	Fragment Diagram	13
3.5	Use Case Diagram	18
4.1	Class Diagram	19
4.2	Data Flow Diagram	21
4.3	Start Chart Diagram	24
4.4	Splash Screen	31
4.5	Home Page	32
4.6	Cart	33
4.7	Update Dish	34
4.8	Post Dish	35
4.9	Track	36
4.10	Customer Profile	37

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO PROJECT

The project DELICATESSEN is a mobile based application that allows the chef to handle all the activities online quickly and safely. Using Interactive GUI anyone can quickly learn to use the complete system. The administrator doesn't have to sit and manage the entire activities on paper, and at the same time, the head will feel comfortable to keep check of the whole system. This system will give him power and flexibility to manage the entire system from a single application.

Our proposed system is an online food ordering system that enables ease for the customers. It overcomes the disadvantages of the traditional queue system. Our proposed system is a medium to order online food hassle free from restaurants as well as mess service. This system improves the method of taking the order from customer. The online food ordering system sets up a food menu online and customers can easily place the order as per their wish. Also with a food menu, customers can easily track the orders. For more secured ordering separate accounts are maintained for each user by providing them an ID and a password.

1.2 ORGANIZATION OF THE REPORT

CHAPTER 2

Chapter 2 talks about the literature survey of the project. It contains all the coding languages involved to run the project. The software used for executing and testing the code. Android Studio is an integrated development environment (IDE) for Google Android OS.

An Emulator – a virtual android device that can simulate variety of hardware features. It can then be used for testing purposes.

XML- Extensible Markup Language is a text language that can be used to describe the behavior of programming languages that process them.

CHAPTER 3

Chapter 3 gives the introduction of the project it includes current and proposed system. It tells about the architecture diagram and description of components in the system architecture. It also gives gist about functional, non-functional, hardware and software requirements and also the user interface.

CHAPTER 4

Chapter 4 talks about all the class diagrams, data flow diagrams, state chat diagrams. The algorithm design and pseudo code of important functions. The -screen shots of the different screens and also include brief description about the screen shot.

CHAPTER 5

Chapter 5 talks about software test specification and the test strategies. It also says what you except from testing and where you are performing testing specification of any testing tools (if used) and any constraints required for testing. It includes the test cases pass/fail criteria of each test case.

CONCLUSION AND REFERENCES

1.3 PROPOSED SYSTEM

A feasibility study is a high-level capsule version of the entire System analysis and Design Process. The study begins by classifying the problem definition. Feasibility is to determine if it's worth doing. Once an acceptance problem definition has been generated, the analyst develops a logical model of the system. A search for alternatives is analyzed carefully. There are 3 parts in feasibility study.

- 1) Operational Feasibility
- 2) Technical Feasibility
- 3) Economical Feasibility

1.3.1 OPERATIONAL FEASIBILITY

Operational feasibility is the measure of how well a proposed system solves the problems, and takes advantage of the opportunities identified during scope definition and how it satisfies the requirements identified in the requirements analysis phase of system development. The operational feasibility assessment focuses on the degree to which the proposed development projects fits in with the existing business environment and objectives with regard to development schedule, delivery date, corporate culture and existing business processes. To ensure success, desired operational outcomes must be imparted during design and development. These include such design-dependent parameters as reliability, maintainability, supportability, usability, producibility, disposability, sustainability, affordability and others.

1.3.2 TECHNICAL FEASIBILITY

This involves questions such as whether the technology needed for the system exists, how difficult it will be to build, and whether the firm has enough experience using that technology. The assessment is based on outline design of system requirements in terms of input, processes, output, fields, programs and procedures. This can be qualified in terms of volume of data, trends, frequency of updating in order to give an introduction to the technical system. This is technically feasible. The technical feasibility assessment is focused on gaining an understanding of the present technical resources of the organization and their applicability to the expected needs of the proposed system. It is an evaluation of the hardware and software and how it meets the need of the proposed system.

1.3.3 ECONOMICAL FEASIBILITY

Establishing the cost-effectiveness of the proposed system i.e., if the benefits do not outweigh the costs then it is not worth going ahead. In the fast-paced world today there is a great need of online social networking facilities. Thus, the benefits of this project in the current scenario make it economically feasible. The purpose of the economic feasibility assessment is to determine the positive economic benefits to the organization that the proposed system will provide. It includes quantification and identification of all the benefits expected. This assessment typically involves a cost/benefits analysis.

CHAPTER 2

LITERATURE SURVEY

2.1 NEED OF SYSTEM

The in person ordering of food is an age old method and it consumes a lot of time of a student for such a simple process of ordering food. This method usually requires a student to walk up to the cash counter wait for his turn list out the food items and take a order coupon. Yet the process isn't complete, he still has to wait for his order to be prepared and finally he collects the order.

The Delicatessen app eliminates most of the steps followed by the student in the offline/in-person mode of ordering food as discussed in the previous paragraph, thus saving a lot of time for the student which can then be used for productive things.

The application allows students to pre-order the food through their android devices at any place as long as they are connected to the internet. In the same way the application also allows the chef to connect to the students and monitor the orders thus helping him to be much efficient and effective. Also during this pandemic the need of this system is very much important as this minimize the human contact thus protecting the bio-bubble and helping in minimizing the spread of the virus.

2.2 SIMILAR SYSTEM

2.2.1 Swiggy:

Swiggy is India's largest online food ordering and delivery platform, founded in July 2014. Swiggy is based in Bangalore, India, and as of March 2019, was operating in 100 Indian cities. It is one of 2 biggest online food delivery company the other one being it's arch rival Zomato. In early 2019, Swiggy expanded into general product deliveries under the name Swiggy Stores.

Swiggy Advantages :

- No Minimum Order.
- Fast Delivery.
- Distance Coverage.
- Live Tracking.
- Order Cancellation.

Swiggy Disadvantages :

- It doesn't have in campus food ordering facility.
- Doesn't provide in college specific application.
- Charges fees from both users and chefs.

2.2.2 Zomato:

Zomato is an Indian multinational restaurant aggregator and food delivery company founded by Deepinder Goyal, Pankaj Chaddah, and Gunjan Patidar in 2008. Zomato provides information, menus and user-reviews of restaurants as well as food delivery options from partner restaurants in select cities. As of 2019, the service is available in 24 countries and in more than 10,000 cities.

Zomato Advantages:

- No Minimum Order.
- Fast Delivery.
- Distance Coverage.
- Live Tracking.
- Order Cancellation.

Zomato Disadvantages:

- It doesn't have in campus food ordering facility.
- Doesn't provide in college specific application.
- Charges fees from both users and chefs.

2.3 TECHNOLOGIES AVAILABLE FOR IMPLEMENTATION

2.3.1 SELECTION OF TECHNOLOGY

In order to complete this project, I have utilized android studio as it is an application that I have become familiar with during my coursework. During my coursework, I have used android studio in the creation of numerous projects, and thus, have used this knowledge to create the classes and GUI elements of this one. I also used two different android devices with different operating systems, one being a Samsung Galaxy s4 on Android 4.2.2 Jelly Bean and the other being an Acer iconia one 8 on Android 5.0 Lollipop. These would prove useful during the testing of the application.

An accelerometer measures acceleration. So ever time your phone moves, it accelerates and stops (which is deceleration or negative acceleration). The app in your phone uses this data which is analyzed by your phone's operating system. The OS notices the rhythmic patterns and figures they're steps.

2.3.2 PROS AND CONS OF ANDRIOD

PROS -

- Support for developing in either the emulator or a device means I can quickly diagnose platform specific issues.
- The support for Kotlin and Java is stellar, with projects easily containing both types of code with ease.
- Hot reload support means that I can quickly test changes without waiting for a length build and optimization process.
- Excellent cross platform support means I can develop on macOS, Windows, or Linux without losing functionality between platforms.

CONS -

- Slow performance on older PCs means that you should develop only on the latest hardware.
- Project opening involves a long indexing and warm up process, meaning that a quick peek at an old project can result in a wait of a few minutes.
- Initial build times can be very long, although these have been slowly improving in recent versions of Android Studio.

CHAPTER 3

REQUIREMENT'S & SPECIFICATION

3.1 INTRODUCTION

3.1.1 CURRENT SYSTEM

Current systems are based on the idea of ordering food from restaurants and they are focused on any common person who is willing to order food online from any restaurant nearby him and get it delivered to his house through a delivery person.

They are not college specific and don't provide a interface for students to order food from canteen or food stalls. Therefore this results in completely different use case of the same application. In addition current systems charge some fees both from customer and the restaurant.

3.1.2 PROPOSED SYSTEM

Our proposed system is an online college food ordering system that enables ease for the customers. It overcomes the disadvantages of the traditional queue system. Our proposed system is a medium to order online food hassle free from college canteen as well as food stalls. This system improves the method of taking the order from customer. The online food ordering system sets up a food menu online and customers can easily place the order as per their wish. Also with a food menu, customers can easily track the orders. This system also provides a feature through which customers can send the note to the chef conveying their taste preference.

Also, the proposed system can help chef to notify the user if the requested dish is available or not thus cancelling the request. The system also provides users with live tracking of order status, thus helping the user to go right on time to collect their orders.

3.2 SYSTEM ARCHITECTURE

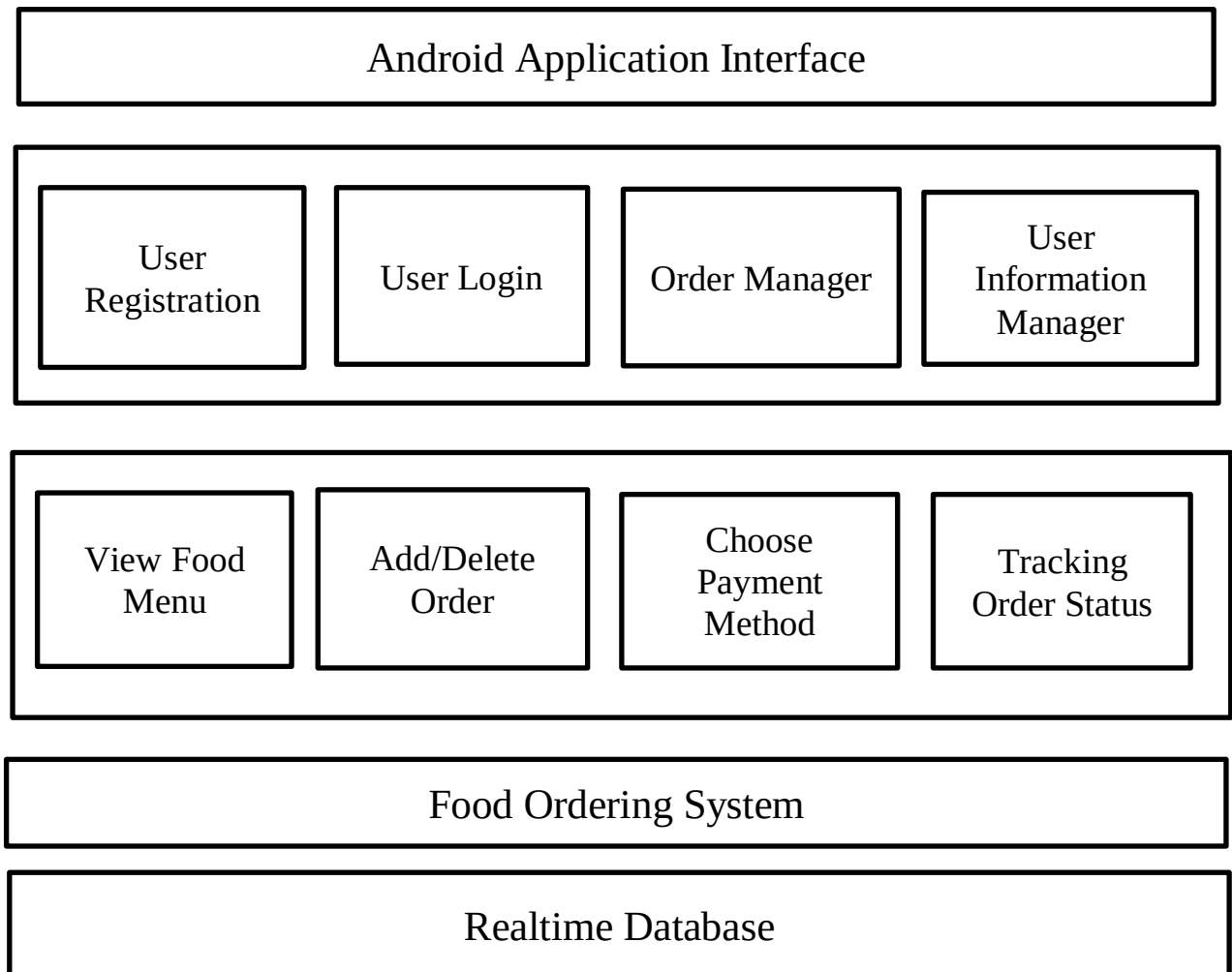


Figure 3.1 System architecture diagram

A system architecture is a representation of a system in which there is a mapping of functionality onto hardware and software components, a mapping of the software architecture onto the hardware architecture, and human interaction with these components.

FUNCTIONAL REQUIREMENTS

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks. So, it's important to make them clear both for the development team and the stakeholders. Generally, functional requirements describe system behavior under specific conditions.

Functional requirements can be classified according to different criteria. For example, we can group them on the basis of the function a given feature must perform in the end product. Of course, they would differ depending on the product being developed, but for the sake of an example, the types of functional requirements might be

- Authentication
- Authorization levels
- Compliance to laws or regulations
- External interfaces
- Transaction's process

1. Speed

Shoppers expect lightning speed load times, and Google does too. Speed should be sufficient enough for the user to use it smoothly.

2. Mobile Friendliness

Make sure your responsive implementation is professionally executed; as a clunky mobile experience discounts the quality of the business for the fast paced shopper.

3. Ease of use in checkout flow

Focus on ease of use; ensure you don't introduce any unnecessary steps to complete the purchase. Strive for a one-click experience.

4. Personalization

A third party personalization tool integrated into your ecommerce platform can influence your sales by up to 59% percent.

5. Accessibility

Follow the universal guidelines provided and implement your accessibility standard to include all shoppers.

The following example shows how functional requirements can be documented:

NO	DESCRIPTION
FR1	Home Page: It allows user to view various dishes and add them to cart.
FR2	Cart: This page shows all the dishes to order at the same place and sums up the total price.
FR3	Post Dish: This allows the chef to post the details of the dish along with its photo to the menu.
FR4	Order Status: Users can view the status of their order.

Table 3.2 Functional requirements table

3.3 NON-FUNCTIONAL REQUIREMENTS

3.3.1 USER INTERFACE

User Interface Design for any application should be very simple. We should have only a few clicks or navigation among the features when using the application to avoid hassle. In this application, there are two main screens, the Login and Home screens. The login page is the first page which appears when the user uses the application. In that page, if he is a new user, he can sign up or if he is an existing user, he can login with the User Credentials. The next screen is the homepage where the users can select features and use the app.

The user interface (UI) for an Android app is built as a hierarchy of layouts and widgets. The layouts are View Group objects, containers that control how their child views are positioned on the screen. Widgets are View objects, UI components such as buttons and text boxes.

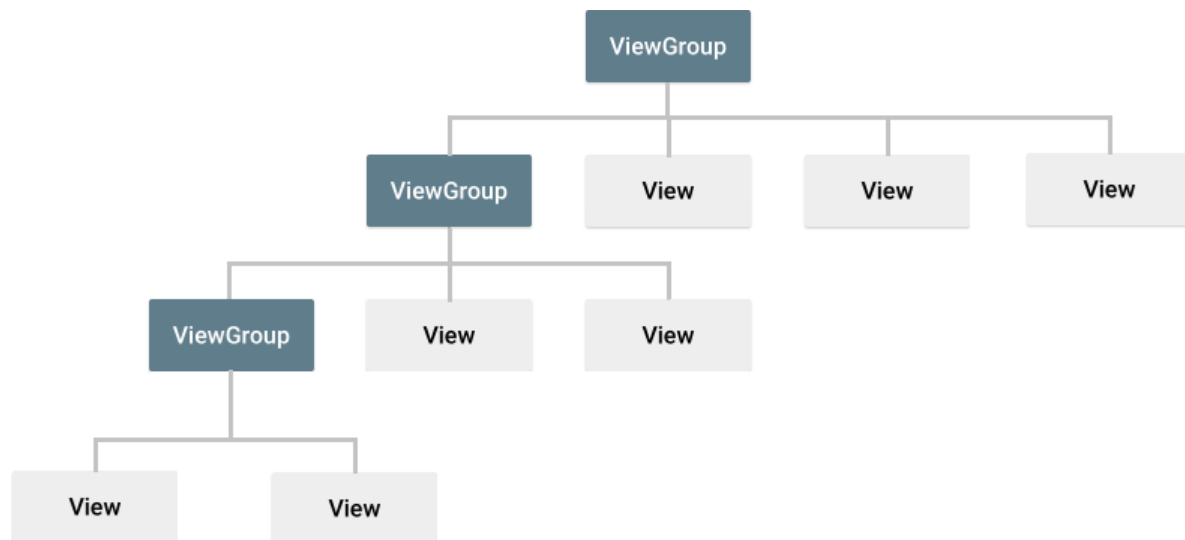


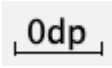
Figure 3.2 User interface diagram

Android provides an XML vocabulary for View Group and View classes, so most of your UI is defined in XML files. However, rather than teach you to write XML, this lesson shows you how to create a layout using Android Studio's Layout Editor. The Layout Editor writes the XML for you as you drag and drop views to build your layout.

Layout Editor

To get started, set up your workspace as follows:

1. In the Project window, open **app > res > layout > activity_main.xml**.
2. To make room for the Layout Editor, hide the **Project** window. To do so, select **View > Tool Windows > Project**, or just click **Project** on the left side of the Android Studio screen.
3. If your editor shows the XML source, click the **Design** tab at the top right of the window.
4. Click  (Select Design Surface) and select **Blueprint**.

5. Click  (**View Options**) in the Layout Editor toolbar and make sure that **Show All Constraints** is checked.
6. Make sure Auto connect is off. A tooltip in the toolbar displays  (**Enable Autoconnection to Parent**) when Auto connect is off.
7. Click  (**Default Margins**) in the toolbar and select **16**. If needed, you can adjust the margins for each view later.
8. Click  (**Device for Preview**) in the toolbar and select **5.5, 1440 × 2560, 560 dpi (Pixel XL)**.

Your Layout Editor now looks as shown in figure.

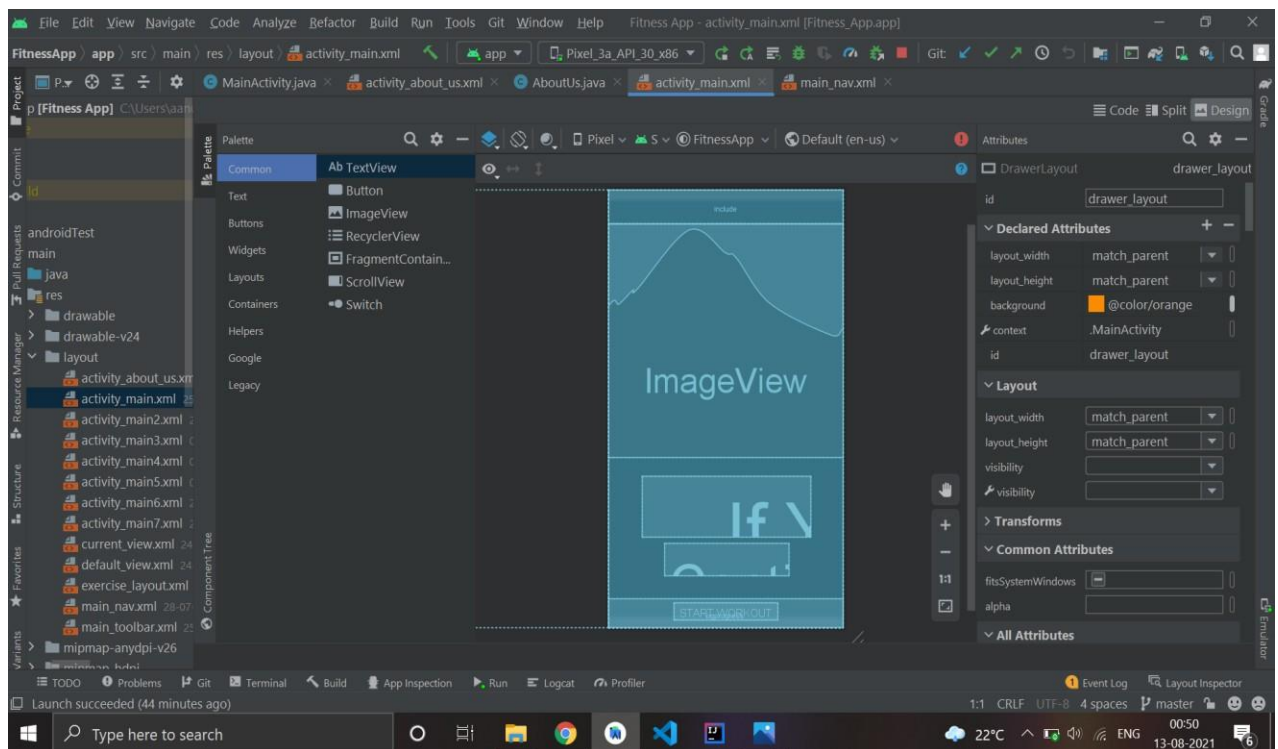


Figure 3.3 XML Diagram

Fragments

A Fragment represents a reusable portion of your app's UI. A fragment defines and manages its own layout, has its own lifecycle, and can handle its own input events. Fragments cannot live on their own--they must be hosted by an activity or another fragment. The fragment's view hierarchy becomes part of, or attaches to, the host's view hierarchy.

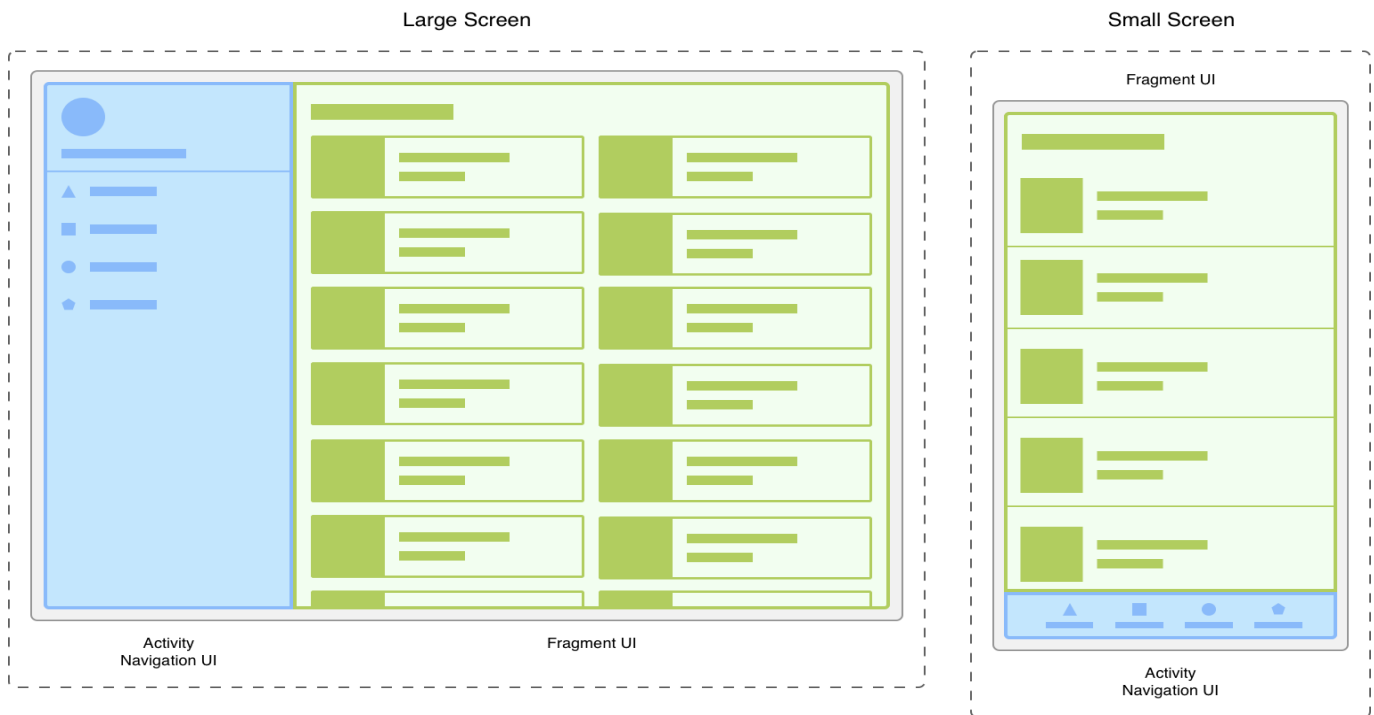


Figure 3.4 Fragment diagram

Figure 1. Two versions of the same screen on different screen sizes. On the left, a large screen contains a navigation drawer that is controlled by the activity and a grid list that is controlled by the fragment. On the right, a small screen contains a bottom navigation bar that is controlled by the activity and a linear list that is controlled by the fragment.

Dividing your UI into fragments makes it easier to modify your activity's appearance at runtime. While your activity is in the **STARTED** lifecycle state or higher, fragments can be added, replaced, or removed.. You can keep a record of these changes in a back stack that is managed by the activity, allowing the changes to be reversed.

You can use multiple instances of the same fragment class within the same activity, in multiple activities, or even as a child of another fragment. With this in mind, you should only provide a fragment with the logic necessary to manage its own UI. You should avoid depending on or manipulating one fragment from another.

3.3.2 Software Requirements

Some of the basic requirements for the development of this project are as follows:

- Operating System : Windows 10
- Tool used : Android Studio
- Coding Language : Java, XML
- Server : Chrome

3.4.2.1 ANDROID STUDIO:

Android Studio is an integrated development environment (IDE) for Google Android Operating System. It is built based on JetBrains' IntelliJ IDEA Community Edition, and it specifically designed for creating applications on Android devices. Some of the key features of Android Studio are as follows:

1. Instant Run – a feature that pushes code and resource changes to the running app. It allows changes to be made to the app without the need to restart the app, or rebuilding the APK, so that the effects can be seen instantly.
2. An Emulator – a virtual android device that can simulate variety of hardware features such as GPS location, network latency, motion sensors, and multi-touch input that can be used to run and install the app. It can then be used for testing purposes.
3. Testing Tools and Frameworks – extensive testing tools such as, JUnit 4 and functional UI test frameworks are included with Android Studio. Espresso Test Recorder can generate UI test code by recording the developer's interactions with the app on a device or emulator. The tests can be run on a device, an emulator, in Firebase Test Lab, or on a continuous integration environment.

3.4.2.2 JAVA PROGRAMMING LANGUAGE:

Java is an object-oriented programming language created by James Gosling, Mike Sheridan, and Patrick Naughton in 1991. In the paper The Java Language Specification Java SE 8 Edition James Gosling states, “Java programming language is a general-purpose, concurrent, class based, object-oriented language. It is designed to be simple enough that many programmers can achieve fluency in the language.

The Java 4 programming language is related to C and C++ but is organized rather differently, with a number of aspects of C and C++ omitted and a few ideas from other languages included. It is intended to be a production language, not a research language.” Java is a very flexible programming language which is used to create many different types of applications for many different operating systems. This is possible because Java can be run on any operating system, as long as the Java Runtime Environment is available. The application created for Android devices must be coded using Java programming language.

3.4.2.1 XML:

XML or Extensible Markup Language is a text language that can be used to describe the behavior of programming languages that process them. XML was developed XML working group in 1996. According to World Wide Web Consortium there are ten design goals for XML.

These design goals are:

1. XML shall be straightforwardly usable over the Internet.
2. XML shall support a wide variety of applications.
3. XML shall be compatible with SGML.
4. It shall be easy to write programs which process XML documents.
5. The number of optional features in XML is to be kept to the absolute minimum, ideally
6. XML documents should be human-legible and reasonably clear.
7. The XML design should be prepared quickly.
8. The design of XML shall be formal and concise.

XML is used when transferring data from the database to the client, and in designing the visual aspect of Android applications. When data is sent from the database, it is sent using

XML. This allows the data to be processed by any programming language the same way, since the data is always sent using XML. As mentioned, XML is also used to design the user interface of Android applications. This means that all the visual aspects such as, the layout of the page, the position of all button and text fields, as well as the color of anything on the page is specified using XML.

3.3.3 Hardware Requirements

- Processor : Intel core i5
- Hard Disk : 20 GB, 80 GB, 160 GB or above
- Monitor : 15 VGA colour, 1024*768 resolution
- RAM : 4GB or above
- Input Device : Keyboard and Mouse

3.4 SYSTEM MODEL

3.4.1 USE CASE DIAGRAM

A use case diagram is a graphic depiction of the interactions among the elements of a system. A use case is a methodology used in system analysis to identify, clarify, and organize system requirements.

Use case diagrams show business use cases, actors, and the relationships between them. The relationships between actors and business use cases state that an actor can use a certain functionality of the business system. You will not find any information.

In the Unified Modelling Language (UML), a use case diagram can summarize the details of your system's users (also known as actors) and their interactions with the system. To build one, you'll use a set of specialized symbols and connectors. An effective use case diagram can help your team discuss and represent:

- Scenarios in which your system or application interacts with people, organizations, or external systems.
- Goals that your system or application helps those entities (known as actors) achieve.
- They are traceable.
- They can serve as the basis for the estimating, scheduling, and validating effort.
- Easily understandable by business users.

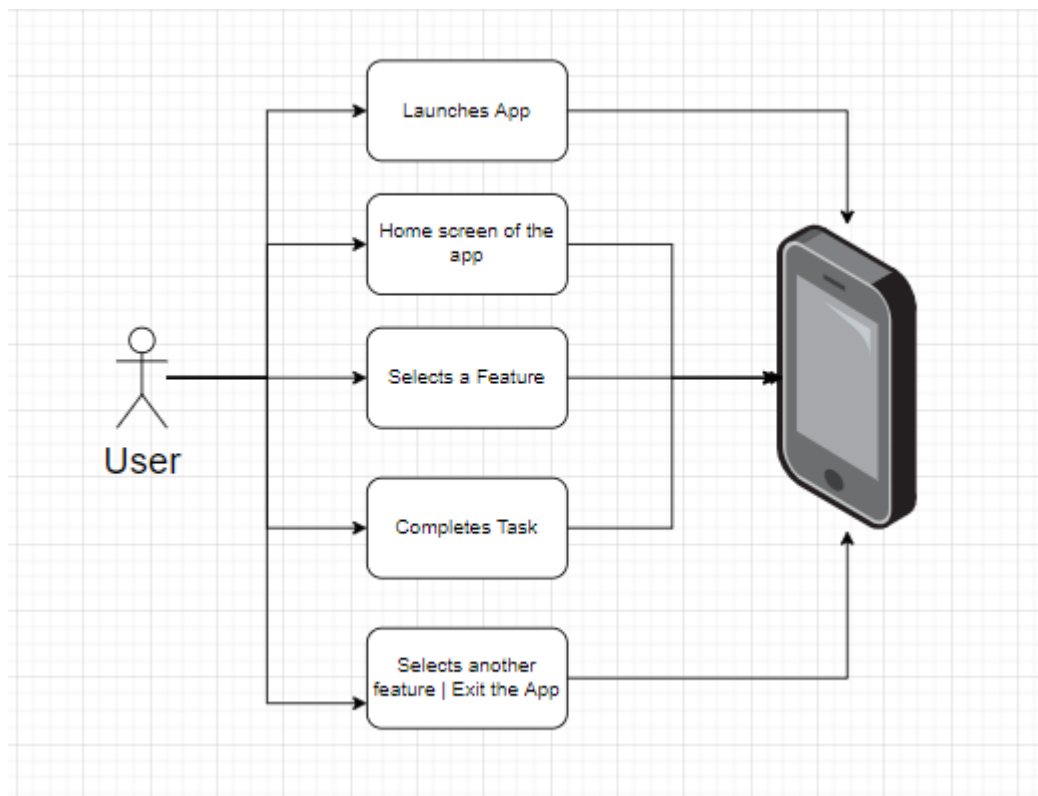


Figure 3.5 Use case diagram

CHAPTER 4

SYSTEM DESIGN

4.1 DATA MODELS

4.1.1 CLASS DIAGRAM

Class diagrams have three main constituents:

- **Class Name** - this is a mandatory section as it contains the name of the class
- **Class Attributes** - this is where all the attributes of a class can be found
- **Class Operations** - includes a list of operations that describe how a class interacts with data

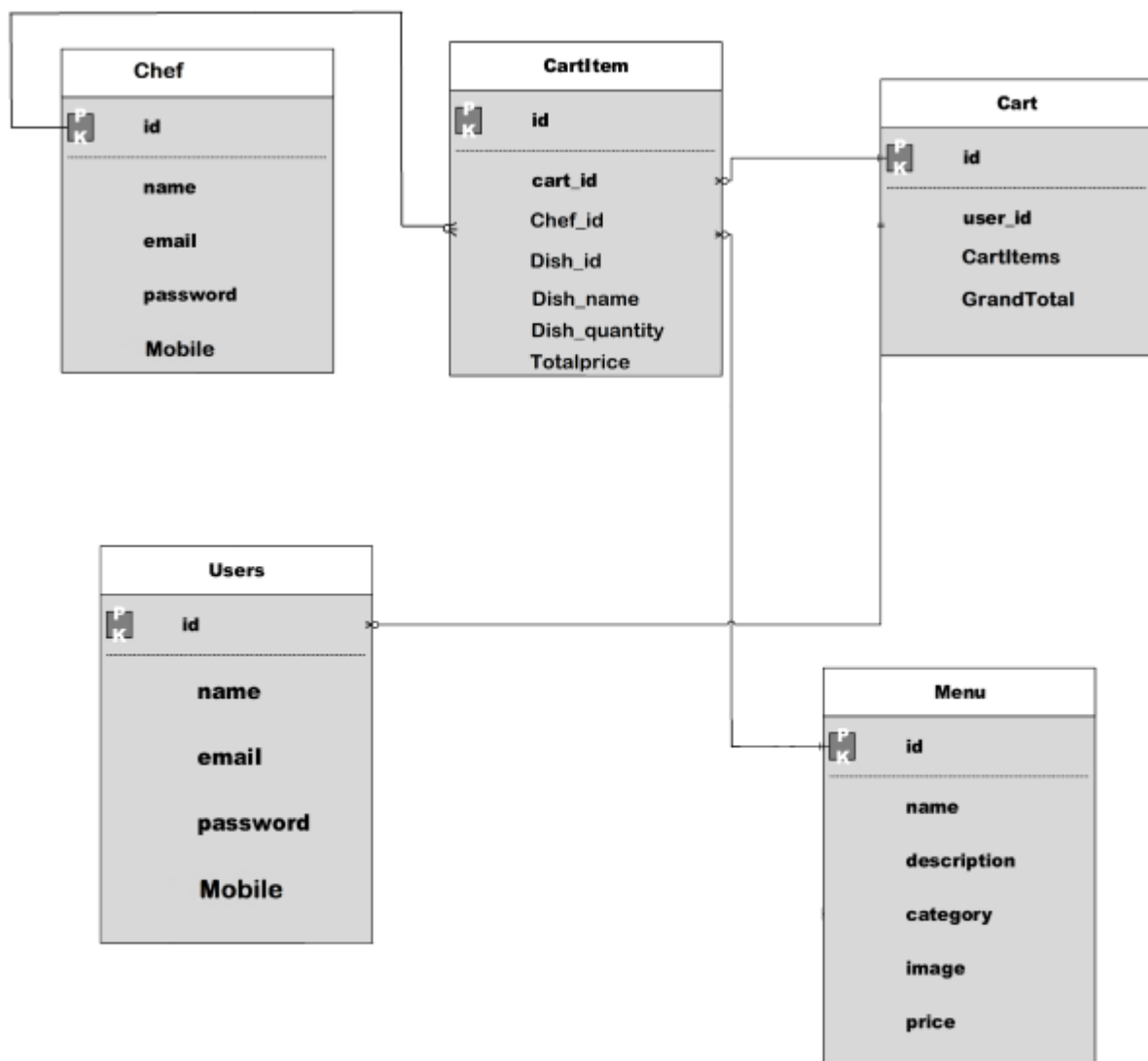


Figure 4.1 Class Diagram of the system.

4.1.2 DATA FLOW DIAGRAM

A data flow diagram (DFD) maps out the flow of information for any process or system. It uses defined symbols like rectangles, circles and arrows, plus short text labels, to show data inputs, outputs, storage points and the routes between each destination. Data flowcharts can range from simple, even hand-drawn process overviews, to in-depth, multi-level DFDs that dig progressively deeper into how the data is handled. They can be used to analyze an existing system or model a new one. Like all the best diagrams and charts, a DFD can often visually “say” things that would be hard to explain in words, and they work for both technical and nontechnical audiences, from developer to CEO. That’s why DFDs remain so popular after all these years. While they work well for data flow software and systems, they are less applicable nowadays to visualizing interactive, real-time or database-oriented software or systems.

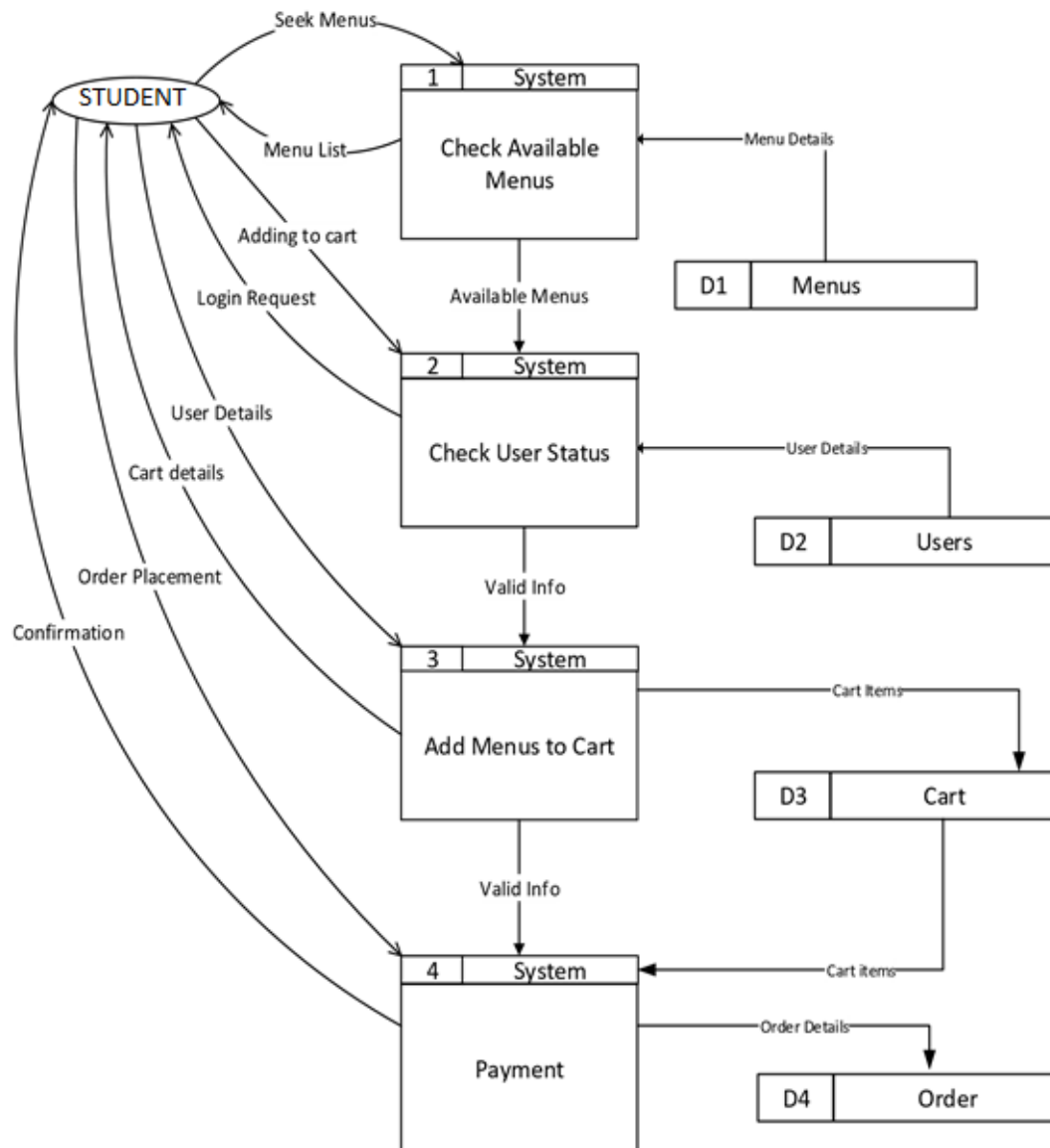


Figure 4.2.1: DFD of processes of customer for ordering food.

Process 1 shows that whenever a visitor request the menus page, the system makes a query in the menu table of the database and acquires available menus and returns the menu list to visitor. If visitor likes to add any menu the menu list (shopping cart), s/he hits “Add to Menu” button. Then in Process 2, the systems checks if the user is logged in. If the user is not logged in, the system requests user to log in by redirecting him/her to the log in page. After getting user’s credential, the system compares the credential against the user database. In Process 3, if the user credential matched with the credentials saved in User table in database, the system adds the menu to Cart table and redirects to the cart page. In Process 4, if user orders the menu, the order details saved in the Order table.

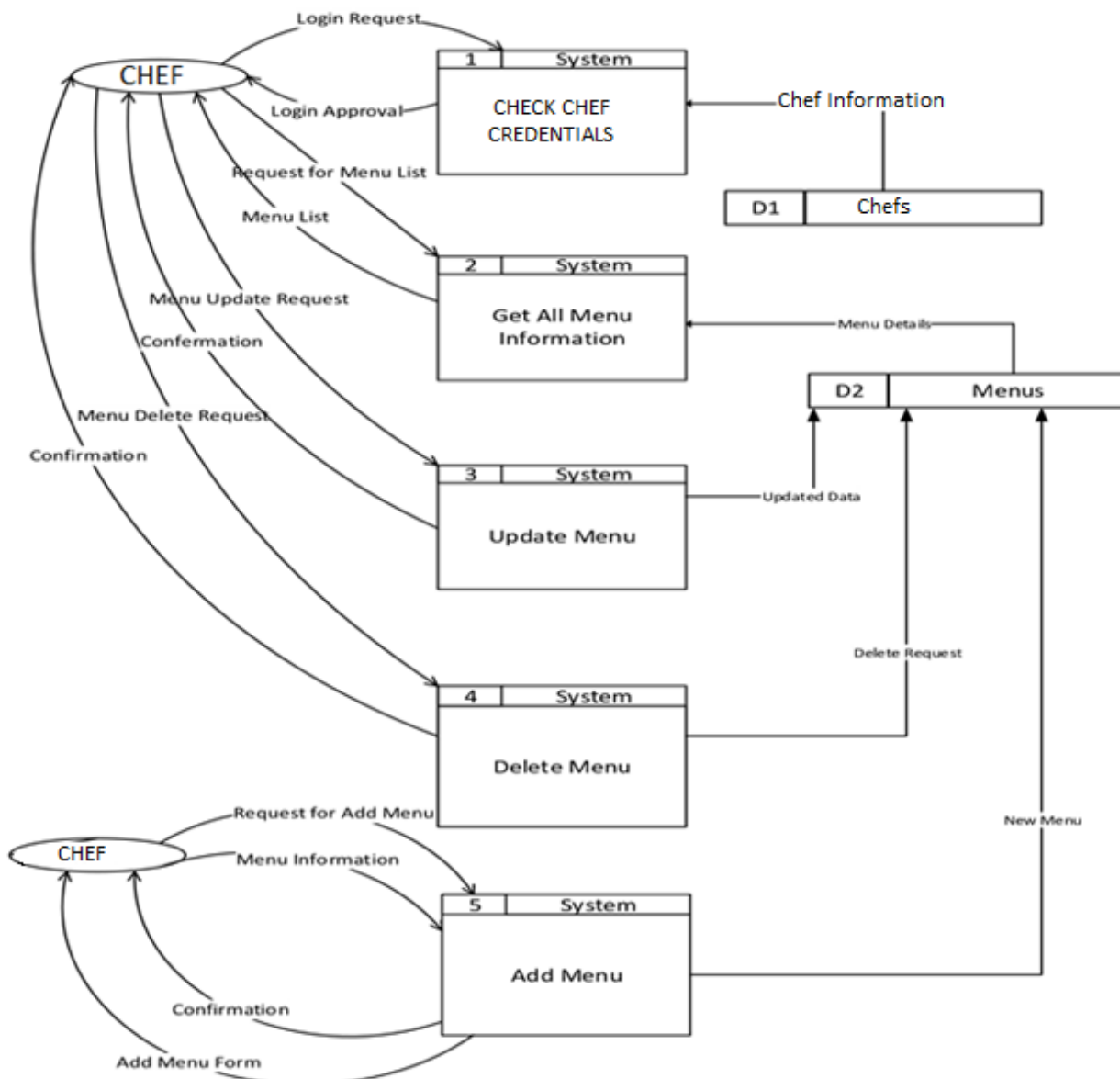


Figure 4.2.2: DFD for processes of Admin.

Figure 4.2.2 illustrates the processes of admin in for manipulating *Menus* table of the database. There are five processes for admin in this application. Process 1 depicts the login of admin to the system. When admin gives credentials and send login request to system, the system checks the credentials against the *Admins* table of the database and gives access. Process 2 illustrates the acquisition of available menu list for admin. In Process 3, the update process of menu has been depicted. In Process 4, deletion of menu has been illustrated. In Process 5, adding process of new menu is shown.

4.1.3 STATE CHART DIAGRAM

State chart diagram is one of the five UML diagrams used to model the dynamic nature of a system. They define different states of an object during its lifetime and these states are changed by events. State chart diagrams are useful to model the reactive systems. Reactive systems can be defined as a system that responds to external or internal event.

State chart diagram describes the flow of control from one state to another state. States are defined as a condition in which an object exists and it changes when some event is triggered.

Following are the main purposes of using State chart diagrams –

- To model the dynamic aspect of a system.
- To model the life time of a reactive system.
- To describe different states of an object during its life time.
- Define a state machine to model the states of an object.

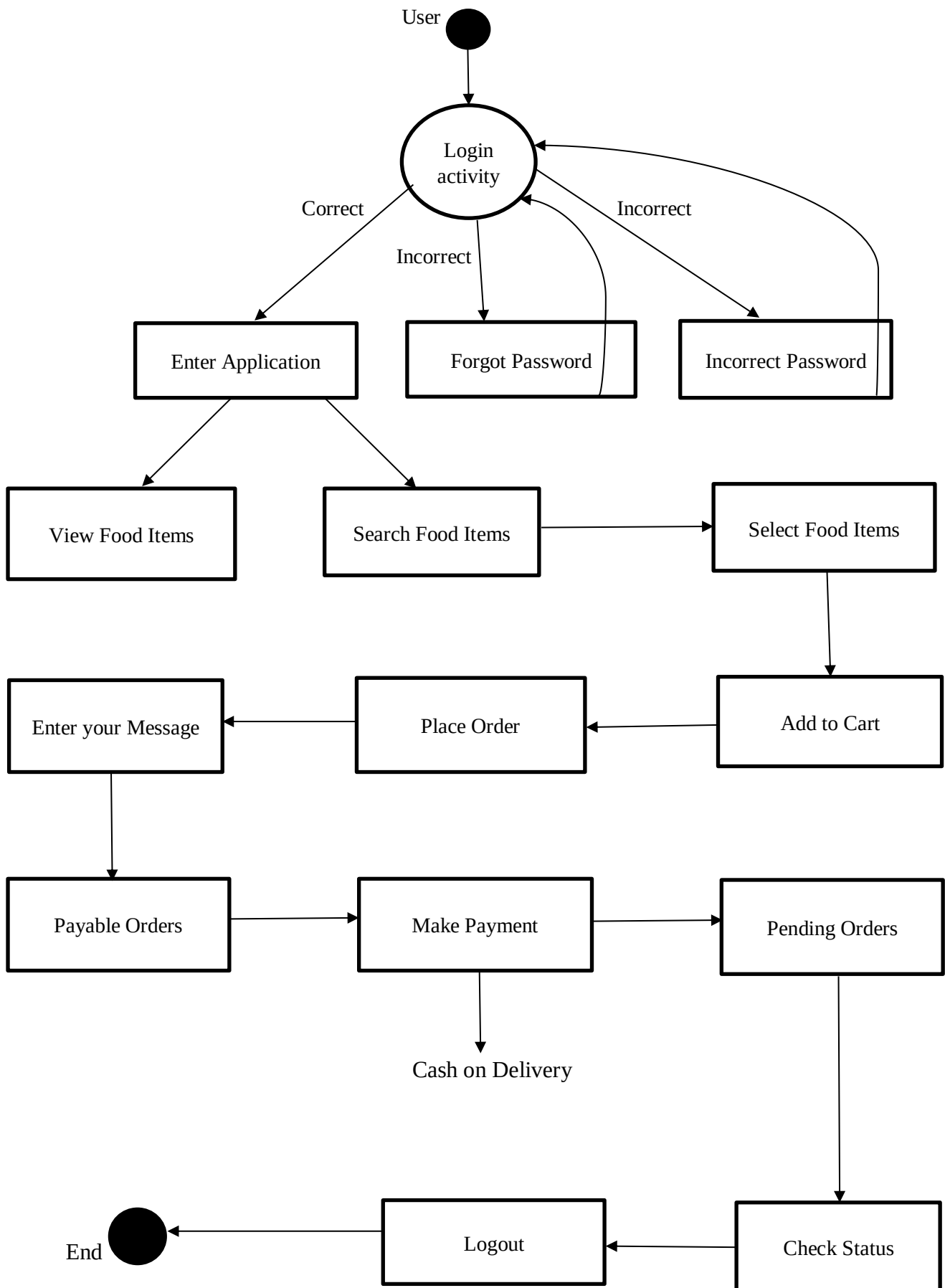


Figure 4.3:State Chart diagram

4.2 ALGORITHM DESIGN

Java Code

Login.java:

```
package com.Delicatessen.app;

import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;

import android.app.ProgressDialog;
import android.content.Intent;
import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import com.Delicatessen.app.R;
import com.Delicatessen.app.ReusableCode.ReusableCodeForAll;
import com.google.android.gms.tasks.OnCompleteListener;
import com.google.android.gms.tasks.Task;
import com.google.android.material.textfield.TextInputLayout;
import com.google.firebase.auth.AuthResult;
import com.google.firebase.auth.FirebaseAuth;

public class Login extends AppCompatActivity {

    TextInputLayout email, pass;
    Button Signin;
    TextView Forgotpassword;
    TextView txt;
    FirebaseAuth FAuth;
    String em;
    String pwd;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);
        try {
            email = (TextInputLayout) findViewById(R.id.Lemail);
            pass = (TextInputLayout) findViewById(R.id.Lpassword);
            Signin = (Button) findViewById(R.id.button4);
            txt = (TextView) findViewById(R.id.textView3);
            Forgotpassword=(TextView)findViewById(R.id.forgotpass);

            FAuth = FirebaseAuth.getInstance();
```

```
SignIn.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        em = email.getText().getText().toString().trim();
        pwd = pass.getText().getText().toString().trim();
        if (isValid()) {

            final ProgressDialog mDialog = new ProgressDialog(Login.this);
            mDialog.setCanceledOnTouchOutside(false);
            mDialog.setCancelable(false);
            mDialog.setMessage("Logging in...");
            mDialog.show();
            FAuth.signInWithEmailAndPassword(em, pwd).addOnCompleteListener(new
OnCompleteListener<AuthResult>() {

                @Override
                public void onComplete(@NonNull Task<AuthResult> task) {
                    if (task.isSuccessful()) {
                        mDialog.dismiss();
                        if (FAuth.getCurrentUser().isEmailVerified()) {
                            Toast.makeText(Login.this, "You are logged in", Toast.LENGTH_SHORT).show();
                            Intent z = new Intent(Login.this, CustomerFoodPanel_BottomNavigation.class);
                            startActivity(z);
                            finish();
                        } else {
                            ReusableCodeForAll.ShowAlert(Login.this, "", "Please Verify your Email");
                        }
                    } else {

                        mDialog.dismiss();
                        ReusableCodeForAll.ShowAlert(Login.this, "Error", task.getException().getMessage());
                    }
                }
            });
        }
    }
});

txt.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {

        Intent Register = new Intent(Login.this, Registration.class);
        startActivity(Register);

    }
});
```

```
Forgotpassword.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        Intent a=new Intent(Login.this,ForgotPassword.class);
        startActivity(a);

    }
});

} catch (Exception e){
    Toast.makeText(this, e.getMessage(), Toast.LENGTH_SHORT).show();
}
}

String emailpattern = "^[a-zA-Z0-9._-]+@eastpoint+\\.+ac+\\.+in+$";
public boolean isValid() {
    email.setErrorEnabled(false);
    email.setError("");
    pass.setErrorEnabled(false);
    pass.setError("");

    boolean isvalidemail=false,isvalidpassword=false,isvalid=false;
    if (TextUtils.isEmpty(em))
    {
        email.setErrorEnabled(true);
        email.setError("Email is required");
    }
    else {
        if (em.matches(emailpattern))
        {
            isvalidemail=true;
        }
        else {
            email.setErrorEnabled(true);
            email.setError("Enter a valid Email Address");
        }
    }

    if (TextUtils.isEmpty(pwd))
    {
        pass.setErrorEnabled(true);
        pass.setError("Password is required");
    }
    else {
        isvalidpassword=true;
    }

    isvalid = (isvalidemail && isvalidpassword) ? true : false;
    return isvalid;
}
}
```

XML Code

AndroidManifest.xml:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.Delicatessen.app">

    <application xmlns:tools="http://schemas.android.com/tools"
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme"
        tools:replace="android:icon">

        <activity
            android:name="com.Delicatessen.app.CustomerFoodPanel_BottomNavigation"
            android:label="HOME" />
        <activity
            android:name="com.Delicatessen.app.ChefFoodPanel.ChefPreparedOrderView"
            android:label="Prepared Orders" />
        <activity android:name=".ChefVerifyPhone"
            android:label="Verify Phone"/>
        <activity
            android:name="com.Delicatessen.app.ChefFoodPanel.ChefPreparedOrder"
            android:label="Prepared Orders" />
        <activity
            android:name="com.Delicatessen.app.ChefFoodPanel.ChefOrdertobePrepareView"
            android:label="Orders to be Prepared" />
        <activity
            android:name="com.Delicatessen.app.ChefFoodPanel.ChefOrderTobePrepared"
            android:label="Orders to be Prepared" />
        <activity
            android:name="com.Delicatessen.app.CustomerFoodPanel.CustomerPayment"
            android:label="Payable Orders" />
        <activity
            android:name="com.Delicatessen.app.CustomerFoodPanel.PayableOrders"
            android:label="Payable Orders" />
        <activity
            android:name="com.Delicatessen.app.CustomerFoodPanel.PendingOrders"
            android:label="Pending Orders" />
        <activity
            android:name="com.Delicatessen.app.ChefFoodPanel.Chef_order_dishes"
            android:label="Orders" />
        <activity
            android:name="com.Delicatessen.app.CustomerFoodPanel.CustomerPhoneSendOTP"
            android:label="Mobile Number" />
        <activity
            android:name="com.Delicatessen.app.CustomerFoodPanel.CustomerPhonenumber"
            android:label="Mobile Number" />
    </application>
</manifest>
```

```
        android:label="Mobile Number" />
    <activity
        android:name="com.Delicatessen.app.CustomerFoodPanel.CustomerForgotpassword"
        android:label="Password" />
    <activity
        android:name="com.Delicatessen.app.CustomerFoodPanel.CustomerPassword"
        android:label="Password" />
    <activity android:name="com.Delicatessen.app.CustomerFoodPanel.OrderDish" />
    <activity
        android:name="com.Delicatessen.app.ChefFoodPanel.Update_Delete_Dish"
        android:label="Home" />
    <activity
        android:name="com.Delicatessen.app.ChefFoodPanel.Chef_PostDish"
        android:label="Post Dish" />
    <activity
        android:name="com.Delicatessen.app.ChefFoodPanel_BottomNavigation"
        android:windowSoftInputMode="adjustNothing" />

    <activity android:name="com.theartofdev.edmodo.cropper.CropImageActivity" />
    <activity
        android:name="com.Delicatessen.app.MainMenu"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.VerifyPhone"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.ChefForgotPassword"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.ChefLogin"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.ChooseOne"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.ChefRegistration"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.ForgotPassword"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.Login"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.Registration"
        android:theme="@style/AppThemeNoActionBar" />
    <activity
        android:name="com.Delicatessen.app.MainActivity"
        android:theme="@style/AppThemeNoActionBar">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
```

```
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>

<service
    android:name="com.Delicatessen.app.SendNotification.MyFirebaseMessagingService"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action android:name="com.google.firebase.MESSAGING_EVENT" />
    </intent-filter>
</service>

<service
    android:name="com.Delicatessen.app.SendNotification.MyFirebaseIdService"
    android:enabled="true"
    android:exported="true">
    <intent-filter>
        <action android:name="com.google.firebase.INSTANCE_ID_EVENT" />
    </intent-filter>
</service>

<meta-data
    android:name="preloaded_fonts"
    android:resource="@array/preloaded_fonts" />
</application>

</manifest>
```

4.3 USER INTERFACE DIAGRAM

4.3.1 Welcome Page

This is the welcome page of this application.

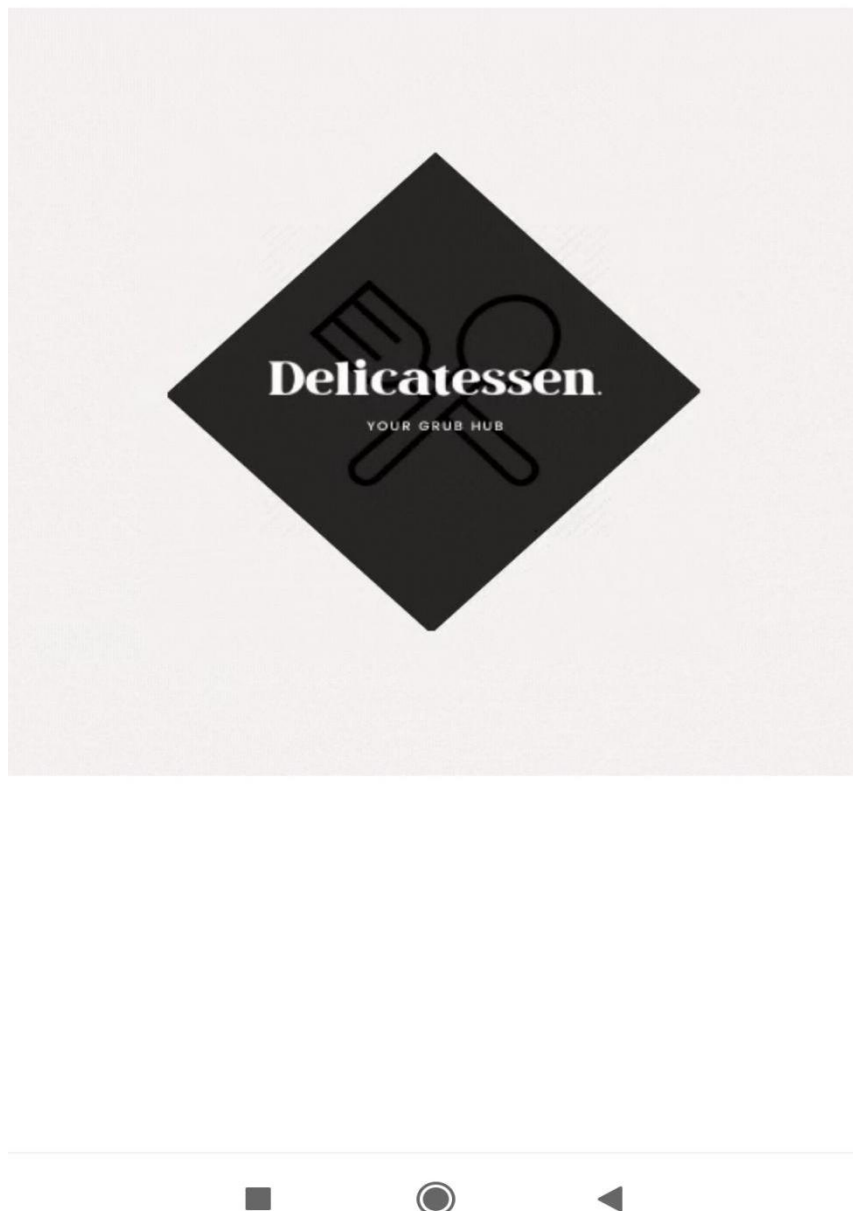


Figure 4.4 Splash Screen

4.3.2 Home Page

It displays the menu to the user and allows user to navigate to other pages through the bottom navigation bar.

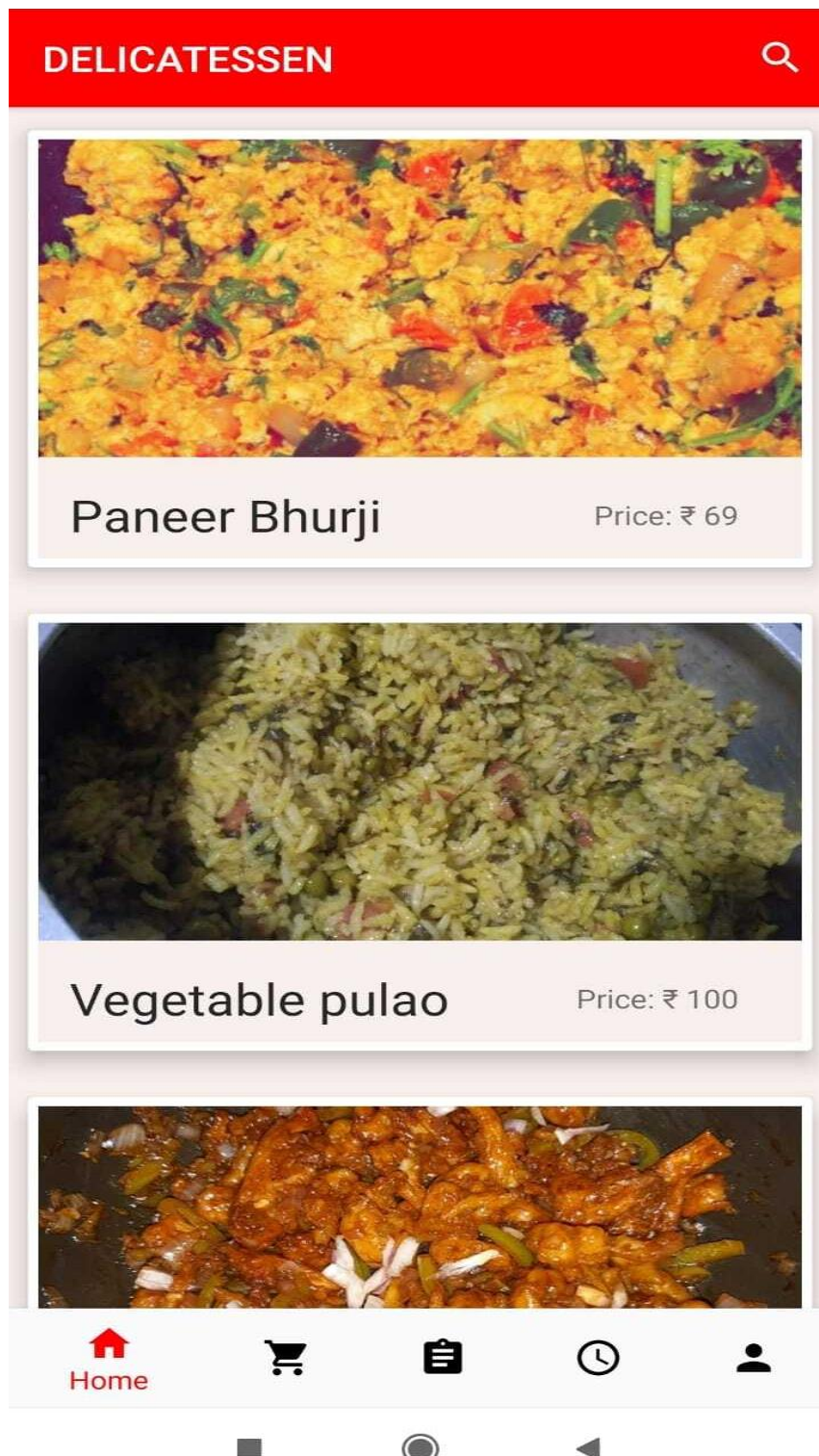


Figure 4.5 Home Page

4.3.3 Cart

This page shows user cart.

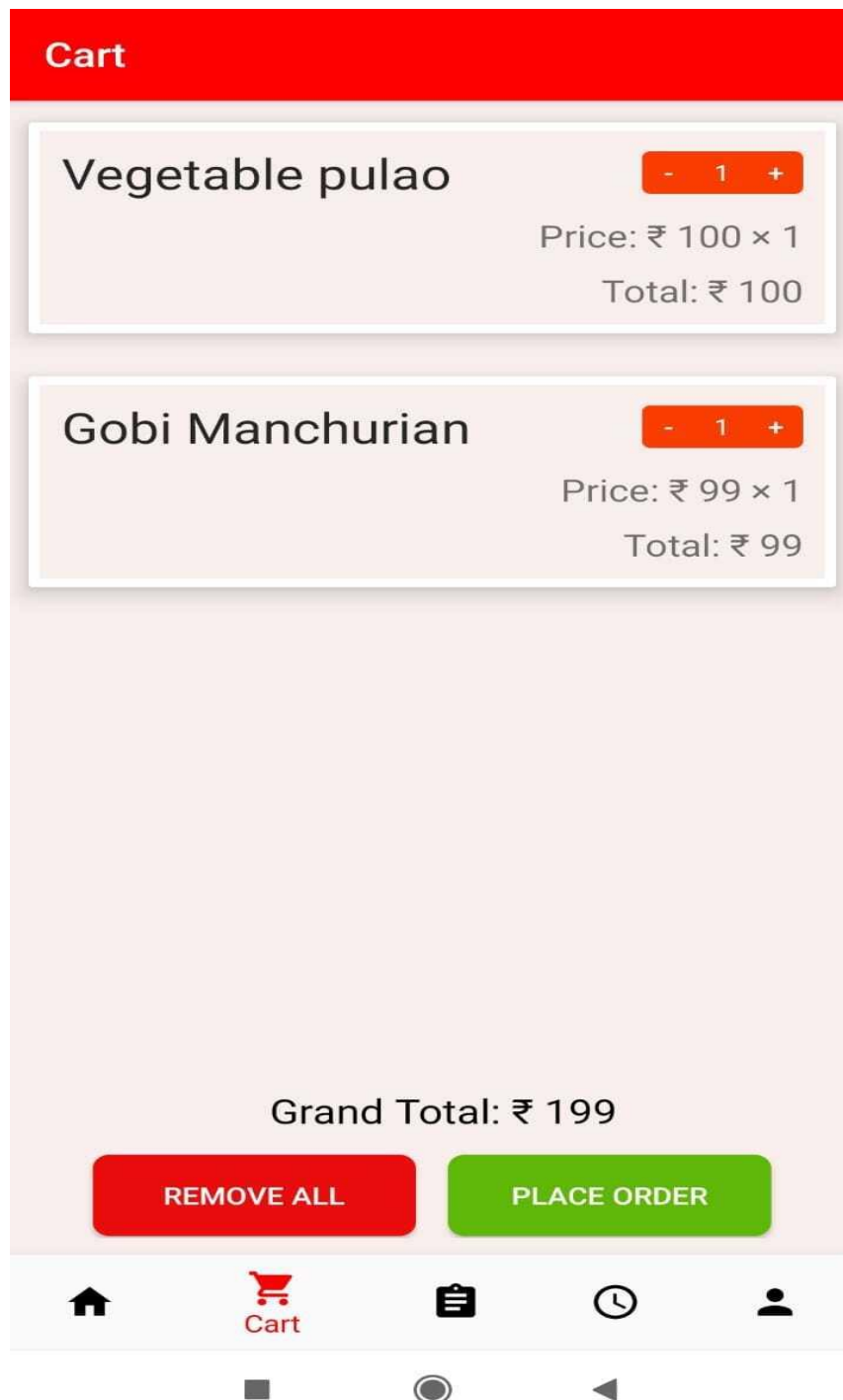


Figure 4.6 Cart

4.3.4 Update Dish

This page allows the chef to update the menu.

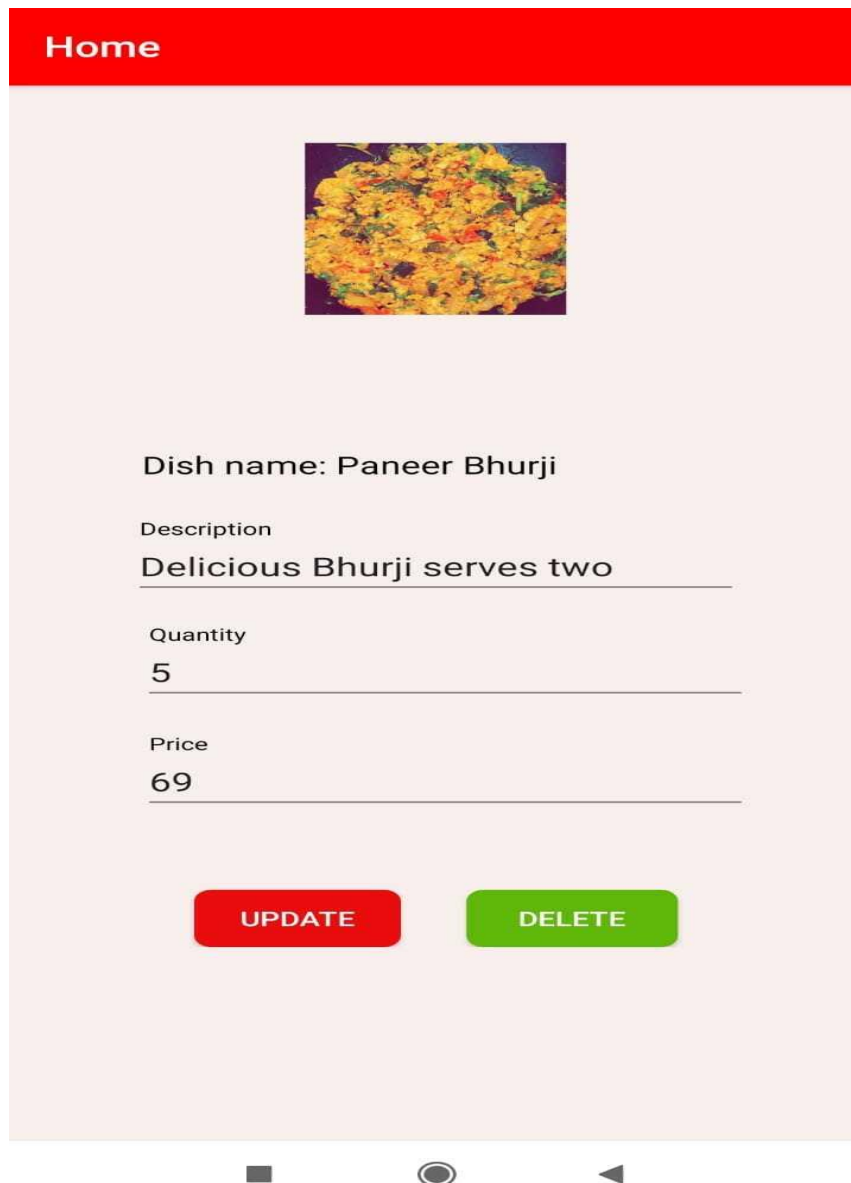
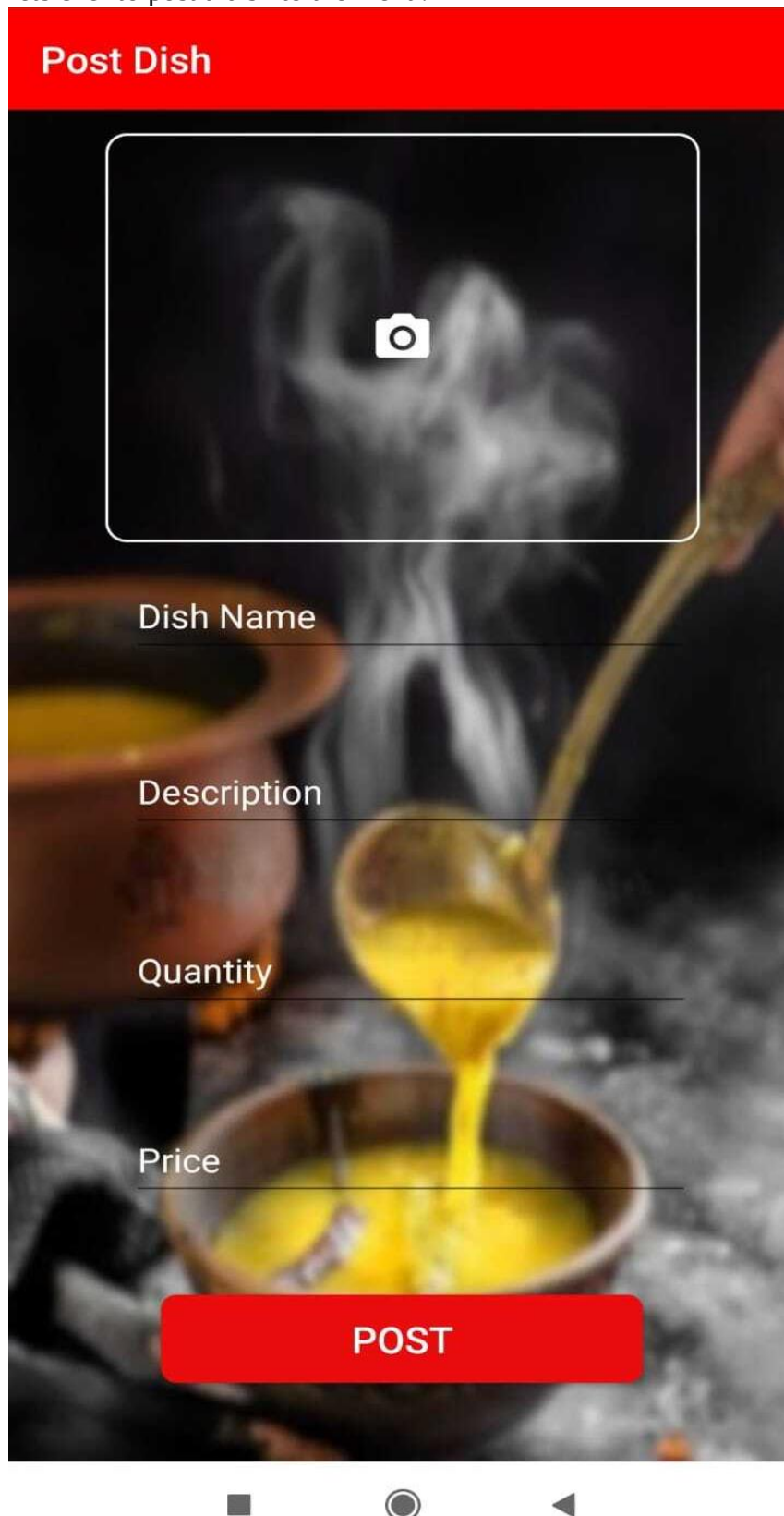


Figure 4.7 Update Dish

4.3.5 Post Dish

This page lets chef to post a dish to the menu.



Post Dish



Dish Name

Description

Quantity

Price

POST

Figure 4.8 Post Dish

4.3.6 Track

This page shows the status of the order to the customer..



Figure 4.9 Track

4.3.7 Customer Profile

This page shows the Customer profile and also allows to update profile

Profile

MY ACCOUNT

FirstName Devnath

LastName Reddy

Mobile no 6363301992

Email 1ep18cs059.cse@eastpoint.ac.in

Password

UPDATE

ACCOUNT ACTIONS

Log Out

Profile

Figure 4.10 Customer Profile

CHAPTER 5

SOFTWARE TEST SPECIFICATION

5.1 GOALS AND CONSTRAINTS

we agreed earlier that you cannot predict the future, but that is exactly what we are going to try to do now! By completing the task, you can begin to give definition to your career goals.

Imagine yourself five years in the future and answer the questions in the present tense as if you have already arrived in your ideal role. Think about what you want to gain from employment and your working environment in general, rather than a specific role.

Write down your answers and you will have an outline picture of where you are aiming in your career. Gaining the perspective is vital to give focus to your efforts and clarity to your plans.

Skills

- What are the industry specific skills you use?
- Do you use financial or analytical skills?
- Are you involved in IT and computers as part of your job ?
- Do you use communicating skills to facilitate or relate ?
- Do you use negotiation or persuasion skills ?
- Are you involved in technical or mechanical tasks?
- Do you use creative skills ?

I use these skills ...

5.2 TESTING STATERGIES

The following outlines the types of testing that will be done for unit, integration, and system testing. While it includes what will be tested, the specific use cases that determine how the testing is done will be detailed in the test design document. The test cases that will be used for designing use cases is shown below.

5.2.1 Unit Testing

In computer programming, unit testing is a software testing method by which individual units of source code, sets of one or more computer program modules together with associated control data, usage procedures, and operating procedures, are tested to determine whether they are fit for use. Intuitively, one can view a unit as the smallest testable part of an application. In procedural programming, a unit could be an entire module, but it is more commonly an individual function or procedure. In object-oriented programming, a unit is often an entire interface, such as a class, but could be an individual method. Unit tests are short code fragments created by programmers or occasionally by white box testers during the development process. It forms the basis for component testing. Ideally, each test case is independent from the others. Substitutes such as method stubs, mock objects, fakes, and test harnesses can be used to assist testing a module in isolation. Unit tests are typically written and run by software developers to ensure that code meets its design and behaves as intended.

Test Case ID: 01				
Name of the Module/Unit:				
Input	Expected Result	Actual Result	Pass/Fail	Remarks
START	Splash screen appears	Splash Screen appears	Pass	-

Table 5.1 Unit Test Table for the Splash Screen

Test Case ID: 02				
Name of the Module/Unit: Login Page				
Input	Expected Result	Actual Result	Pass/Fail	Remarks
Enter username and password	Login successfully	Login successfully	Pass	-

Table 5.2 Unit Test Table for the Login Page

5.2.2 Integration Testing

Integration testing (sometimes called integration and testing, abbreviated I&T) is the phase in software testing in which individual software modules are combined and tested as a group. It occurs after unit testing and before validation testing. Integration testing takes as its input modules that have been unit tested, groups them in larger aggregates, applies tests defined in an integration test plan to those aggregates, and delivers as its output the integrated system ready for system testing.

Functional Requirements No.	Functional Requirements	Action	Expected Result	Actual Result	Pass/Fail	Remarks
FR1	Signup Page	Enter your Signup details	Signup Successful	Signup Successful	Pass	-
FR2	Login Page	Enter your Login details	Logged in successfully	Logged in successfully	Pass	-
FR3	Home Page	View Dishes	Dishes displayed	Dishes displayed	Pass	-
FR4	Pending Orders Page	Display pending orders	Pending Orders displayed	Pending Orders displayed	Pass	-
FR5	Post dish page	Post Dish to the menu	Dish Posted to the menu	Dish Posted to the menu	Pass	-
FR6	Cart Page	Display Cart item	Displays Cart items	Displays Cart items	Pass	-
FR7	Order Status Page	View order status	Order status displayed	Order status displayed	Pass	-
FR8	Customer Profile Page	View Customer profile	Customer profile displayed	Customer profile displayed	Pass	-

Table 5.2 Test case table

5.2.3 SYSTEM TESTING

System testing of software or hardware is testing conducted on a complete, integrated system to evaluate the system's compliance with its specified requirements. System testing falls within the scope of black-box testing, and as such, should require no knowledge of the inner design of the code or logic. As a rule, system testing takes, as its input, all of the "integrated" software components that have passed integration testing and also the software system itself integrated with any applicable hardware system(s).

The purpose of integration testing is to detect any inconsistencies between the software units that are integrated together (called assemblages) or between any of the assemblages and the hardware. System testing is a more limited type of testing; it seeks to detect defects both within the "inter-assemblages" and also within the system as a whole.

System testing is performed on the entire system in the context of a Functional Requirement Specification(s) (FRS) and/or a System Requirement Specification (SRS). System testing tests not only the design, but also the behavior and even the believed expectations of the customer. It is also intended to test up to and beyond the bounds defined in the software/hardware requirements specification(s).

CONCLUSION

This report gives an insight into the background as to why I chose to develop this application. I believed that there was a problem with apps of this type and that if done a certain way this could be solved. The idea and my motivations have been stated clearly. The requirements are described in detail and the technologies used have also been outlined in this document. I have touched on the Systems design and architecture and provided a class diagram to explain it further. I believe that I achieved most of what I have set out to do when this idea was conceived. However, there are multiple different areas which could be improved and lots of room for expansion.

Finally, in Delicatessen (Online College Food Ordering system), we have developed secure, user-friendly food Ordering Management System for our college campus. This System can take care of each member whether it is an Chef or Customer. This System will help chef to properly manage the food menu and manage the orders of the customers too without creating any hassle. This System is completely secure since every user is provided with user ID and Password so there is no chance of any unauthorized access.

So, using this system will help in reducing the Time consumed and provide more facility for user to like the services.

REFERENCES

1. Patil, Pratek and Karl Moss, 2017, Java Database Programming with JDBC, Coriclis Group Books.
2. Sierra, Kathy and Bert Bates, 2003, Head First Java, O'Reilly.
3. Sarcar, Vaskaran, 2018, Java Design Patterns, Apress.
4. <https://www.tutorialspoint.com/java>
5. <http://www.javapoint.com/java-tutorial>
6. <https://developer.android.com/>
7. <https://youtu.be/F82x3gUkF04>
(splash screen tutorial)
8. <https://youtu.be/Btli00YA1eI>
(fragments tutorial)
9. <https://youtu.be/SV9pJqR41KI>
(firebase tutorial)